```
iex> send(pid, {:add, 1, 2})
3

iex> send(pid, {:add, 50, 50})
100
```

You could extend this exercise by turning loop/0 into loop/1 and keeping track of state for each message. If you did this, you would have a server that changes state based on messages it has received from the outside world. This is a pretty small change but is still pretty cumbersome. Don't worry, GenServer will make this easy for us too.

Before we get into GenServer, let's look at some of the interesting details of how processes are implemented. These may seem unimportant at first, but they drastically shape the runtime characteristics of an Elixir application.

## Error Isolation in Processes

There's an argument to be made that the genius of the BEAM is not its parallel execution ability but, rather, its ability to isolate errors. Let's put that into perspective: if two requests come into a web server at the same time, and one of the requests crashes, then we wouldn't expect the other request to also crash.

Let's spawn two processes and then crash one of them to create a basic demonstration:

```
$ iex -S mix
iex> p1 = Examples.Spawn.Infinite.start()
#PID<0.161.0>

iex> p2 = Examples.Spawn.Infinite.start()
#PID<0.163.0>

iex> send(p1, :crash)
[error] Process #PID<0.161.0> raised an exception
** (RuntimeError) I crashed

iex> [Process.alive?(p1), Process.alive?(p2)]
[false, true]
```

This is a simple example, but it serves to demonstrate that we didn't have to do anything to isolate this error. In fact, it wouldn't be possible to crash one of these processes from the other. Of course, an event like a database failure is going to cause errors all over an application, but that would be due to an external factor rather than an internal one.

It's easy to take error isolation for granted. Frameworks in languages that don't provide error isolation use clever programming to make it feel like there's